

Verification of Functional Programs

Andrés Sicard-Ramírez

Universidad EAFIT

Semester 2026-1

Pacto Pedagógico

Como miembros de la Universidad EAFIT, nos comprometemos a actuar de manera íntegra siguiendo los más altos estándares éticos y morales.

- Respeto
- Tolerancia
- Honradez
- Compromiso

Página web del curso

<https://asr.github.io/courses/verification-of-functional-programs/2026-1>

Pacto Pedagógico

Página web del curso

<https://asr.github.io/courses/verification-of-functional-programs/2026-1>

Conducto regular, fechas y porcentajes de las evaluaciones

La información está en la página web del curso.

Pacto Pedagógico

Página web del curso

<https://asr.github.io/courses/verification-of-functional-programs/2026-1>

Conducto regular, fechas y porcentajes de las evaluaciones

La información está en la página web del curso.

Responsabilidad compartida

- Profesor
- Estudiantes

Asistencia a clase

Reglamento académico de los programas de posgrado, Capítulo VI, Artículo 62, Parágrafo 2.

“El estudiante de posgrado cuyas faltas de asistencia lleguen al treinta por ciento (30%) del total de las horas de clase programadas para el curso o para una parte de éste, cuando se desarrolle con más de un profesor, en secciones temáticas denominadas ‘módulo’, pierde con calificación de cero punto cero (0.0) del seminario o curso correspondiente y esta nota afecta el promedio crédito acumulado.”

Pacto Pedagógico

Orientaciones para el curso

- Se recomienda cuatro horas de trabajo por semana (dos horas por cada hora de clase).
- Las clases son presenciales.
- La evaluación a la docencia es obligatoria.
- Se recomienda revisar periódicamente los canales de comunicación institucionales (EAFIT Interactiva y el correo institucional).
- El estudiante podrá entrar a clase a más tardar 20 minutos después de la hora asignada para su inicio.

Interactive Proof Assistants

Description

*“Proof assistants are computer systems that allow a user to do mathematics on a computer, but not so much the computing (numerical or symbolical) aspect of mathematics but the aspects of **proving** and **defining**. So a user can **set up** a mathematical theory, define properties and do logical reasoning with them.”*
(Geuvers 2009, p. 3.)

Interactive Proof Assistants

Description

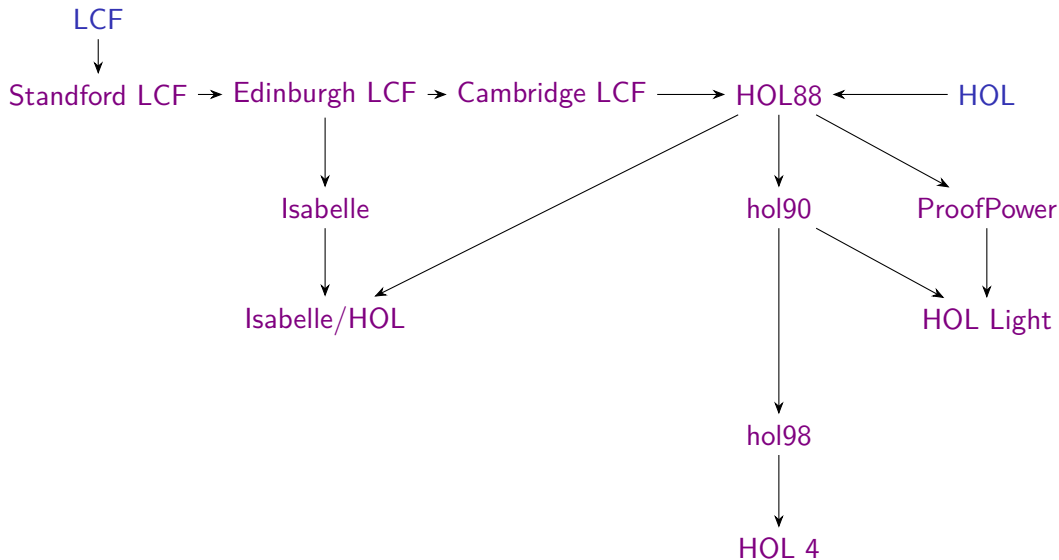
*“Proof assistants are computer systems that allow a user to do mathematics on a computer, but not so much the computing (numerical or symbolical) aspect of mathematics but the aspects of **proving** and **defining**. So a user can **set up** a mathematical theory, define properties and do logical reasoning with them.”*
(Geuvers 2009, p. 3.)

Examples

- Based on set theory: Isabelle/ZFC, Metamath and Mizar
- Based on higher-order logic: HOL4, HOL Light and Isabelle/HOL
- Based on type theories: Agda, Rocq and Lean.

- University of Cambridge (England) and Technical University of Munich (German)
- Based on higher-order logic
- Tactic-based
- Extraction of programs to **Haskell**, **OCaml**, **Scala** and **SML**
- Written in **SML**
- Integration with ATPs and SMT solvers

LCF and HOL Proof Checkers and Proof Assistants



*"It seems very natural to extend the system F of first-order logic by permitting quantification on predicate, propositional, and function variables as well as individual variables. One thus obtains the wffs of a system of **second-order logic**. One might then introduce predicate and function variables of higher type to denote relations and functions whose arguments may be relations and functions of individuals as well as individuals. Thus one would be led to a system of **third-order logic**, and if one permitted quantification with respect to these new variables, one would obtain a system of **fourth-order logic**. This process can be continued indefinitely to obtain logics of arbitrarily high order. Of course, after a while one runs out of different types of letters to use for different types of variables, but this problem can be solved by introducing **type symbols** to indicate the types of variables, and using a letter with type symbol α as subscript for a variable of type α ." (Andrews [1986] 2002, p. 201)*

HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.



HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.

Also called “simple type theory” or “Church’s type theory”.



HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.

Also called “simple type theory” or “Church’s type theory”.

Types: Base types (ι of individuals and o of truth values), variables have fixed types, function types.



HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.

Also called “simple type theory” or “Church’s type theory”.

Types: Base types (ι of individuals and o of truth values), variables have fixed types, function types.

Terms: Variables, constants, λ -abstractions and function application.



HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.

Also called “simple type theory” or “Church’s type theory”.

Types: Base types (ι of individuals and o of truth values), variables have fixed types, function types.

Terms: Variables, constants, λ -abstractions and function application.

First-order logic with equality (formulae are of type o).



HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.

Also called “simple type theory” or “Church’s type theory”.

Types: Base types (ι of individuals and o of truth values), variables have fixed types, function types.

Terms: Variables, constants, λ -abstractions and function application.

First-order logic with equality (formulae are of type o).

Sets: Sets of individuals with type $\iota \rightarrow o$, sets of sets of individuals with type $(\iota \rightarrow o) \rightarrow o$, etc.



HOL (Higher-Order Logic)

Alonzo Church (1940). “A Formulation of the Simple Theory of Types”.

Also called “simple type theory” or “Church’s type theory”.

Types: Base types (ι of individuals and o of truth values), variables have fixed types, function types.

Terms: Variables, constants, λ -abstractions and function application.

First-order logic with equality (formulae are of type o).

Sets: Sets of individuals with type $\iota \rightarrow o$, sets of sets of individuals with type $(\iota \rightarrow o) \rightarrow o$, etc.



“Church’s type theory has been extensively studied by two of Church’s students, L. Henkin and P. Andrews. . . Henkin also showed in [30] that Church’s type theory could be reformulated using only four primitive notions: function application, function abstraction, equality, and definite description. . . Andrews formulated a version of Church’s type theory called Q_0 that employs the ideas developed by Church, Henkin, and himself.” (Farmer 2008, p. 269)

LCF (Logic for Computable Functions)

Dana S. Scott ([1969] 1993). “A Type-Theoretical Alternative to ISWIM, CUCH, OWHY”.

Typed combinatory logic for reasoning about computable (continuous) functions including partial and non-terminating ones.

Types are interpreted via domain theory.

Recursive functions are interpreted via least-fixed points.



Robin Milner (1972). “Logic for Computable Functions. Description of a Machine Implementation”.

Proof checker for LCF.



Edinburgh LCF

Michael J. Gordon, Robin Milner and Christopher P. Wadsworth (1979). “Edinburgh LCF. A Mechanised Logic of Computation”.

Edinburgh LCF

Michael J. Gordon, Robin Milner and Christopher P. Wadsworth (1979). “Edinburgh LCF. A Mechanised Logic of Computation”.

A problem with Stanford LCF was that the size of proofs was limited by available memory.

Edinburgh LCF

Michael J. Gordon, Robin Milner and Christopher P. Wadsworth (1979). “Edinburgh LCF. A Mechanised Logic of Computation”.

A problem with **Stanford LCF** was that the size of proofs was limited by available memory.

For solving the above problem:

“He [Milner] had the idea that instead of saving whole proofs, the system should just remember the results of proofs, namely theorems. . . To ensure that theorems could only be created by proof, Milner had the brilliant idea of using an abstract data type whose predefined values were instances of axioms and whose operations were inference rules. Strict typechecking then ensured that the only values that could be created were those that could be obtained from axioms by applying a sequence of inference rules—namely theorems.” (M. J. C. Gordon 2000, p. 170)

Edinburgh LCF

Michael J. Gordon, Robin Milner and Christopher P. Wadsworth (1979). “Edinburgh LCF. A Mechanised Logic of Computation”.

A problem with **Stanford LCF** was that the size of proofs was limited by available memory.

For solving the above problem:

“He [Milner] had the idea that instead of saving whole proofs, the system should just remember the results of proofs, namely theorems. . . To ensure that theorems could only be created by proof, Milner had the brilliant idea of using an abstract data type whose predefined values were instances of axioms and whose operations were inference rules. Strict typechecking then ensured that the only values that could be created were those that could be obtained from axioms by applying a sequence of inference rules—namely theorems.” (M. J. C. Gordon 2000, p. 170)

Introduction of the functional programming language ML (Meta Language).

Michael J. Gordon, Robin Milner and Christopher P. Wadsworth (1979). “Edinburgh LCF. A Mechanised Logic of Computation”.

A problem with **Stanford LCF** was that the size of proofs was limited by available memory.

For solving the above problem:

“He [Milner] had the idea that instead of saving whole proofs, the system should just remember the results of proofs, namely theorems. . . To ensure that theorems could only be created by proof, Milner had the brilliant idea of using an abstract data type whose predefined values were instances of axioms and whose operations were inference rules. Strict typechecking then ensured that the only values that could be created were those that could be obtained from axioms by applying a sequence of inference rules—namely theorems.” (M. J. C. Gordon 2000, p. 170)

Introduction of the functional programming language ML (Meta Language).

Introduction of the concepts of “tactics” and “tacticals”.

Lawrence C. Paulson ([1987] 2003). “Logic and Computation. Interactive Proof with Cambridge LCF”.

Cambridge LCF

Lawrence C. Paulson ([1987] 2003). “Logic and Computation. Interactive Proof with Cambridge LCF”.

Many theoretical and technical improvements in relation to Cambridge LCF.

Mike Gordon (1985). “HOL. A Machine Oriented Formulation of Higher Order Logic”.

Mike Gordon (1985). “HOL. A Machine Oriented Formulation of Higher Order Logic”.

“The design of HOL was largely taken ‘off the shelf’, the theory being classical higher order logic and the implementation being LCF.” (M. J. C. Gordon 2000, p. 174)

Mike Gordon (1985). “HOL. A Machine Oriented Formulation of Higher Order Logic”.

“The design of HOL was largely taken ‘off the shelf’, the theory being classical higher order logic and the implementation being LCF.” (M. J. C. Gordon 2000, p. 174)

“In this paper we describe a formal language intended as a basis for hardware specification and verification. This language is not new; the only originality in what follows lies in the presentation of details.” (M. J. C. Gordon 1985, p. 2)

Mike Gordon (1985). “HOL. A Machine Oriented Formulation of Higher Order Logic”.

“The design of HOL was largely taken ‘off the shelf’, the theory being classical higher order logic and the implementation being LCF.” (M. J. C. Gordon 2000, p. 174)

“In this paper we describe a formal language intended as a basis for hardware specification and verification. This language is not new; the only originality in what follows lies in the presentation of details.” (M. J. C. Gordon 1985, p. 2)

The HOL logic is Church’s HOL with two modifications: polymorphism and the Axiom of Choice is built-in via Hilbert’s ϵ -operator.

Primitive definitional principles: *“The HOL system, unlike LCF, emphasises definition rather than axiom postulation as the primary method of developing theories. Higher order logic makes possible a purely definitional development of many mathematical objects (numbers, lists, trees, etc.) and this is supported and encouraged.”* (M. J. C. Gordon 2000, p. 174)

Primitive definitional principles: *“The HOL system, unlike LCF, emphasises definition rather than axiom postulation as the primary method of developing theories. Higher order logic makes possible a purely definitional development of many mathematical objects (numbers, lists, trees, etc.) and this is supported and encouraged.” (M. J. C. Gordon 2000, p. 174)*

Derived definitional principles (high-level principles programmed in ML:

Primitive definitional principles: *“The HOL system, unlike LCF, emphasises definition rather than axiom postulation as the primary method of developing theories. Higher order logic makes possible a purely definitional development of many mathematical objects (numbers, lists, trees, etc.) and this is supported and encouraged.” (M. J. C. Gordon 2000, p. 174)*

Derived definitional principles (high-level principles programmed in ML:

- *“Melham implemented a derived type-definition principle that converts descriptions of recursive datatypes into primitive definitions and then automatically derives the natural induction and primitive recursion principles for the datatype.” (M. J. C. Gordon 2000, p. 174)*

Primitive definitional principles: *“The HOL system, unlike LCF, emphasises definition rather than axiom postulation as the primary method of developing theories. Higher order logic makes possible a purely definitional development of many mathematical objects (numbers, lists, trees, etc.) and this is supported and encouraged.” (M. J. C. Gordon 2000, p. 174)*

Derived definitional principles (high-level principles programmed in ML:

- *“Melham implemented a derived type-definition principle that converts descriptions of recursive datatypes into primitive definitions and then automatically derives the natural induction and primitive recursion principles for the datatype.” (M. J. C. Gordon 2000, p. 174)*
- Melham also implemented a derived definitional principle that *“allows inductively defined relations to be specified by a transition system, and then a rule-induction tactic to be automatically generated” (M. J. C. Gordon 2000, p. 174)*

Primitive definitional principles: *“The HOL system, unlike LCF, emphasises definition rather than axiom postulation as the primary method of developing theories. Higher order logic makes possible a purely definitional development of many mathematical objects (numbers, lists, trees, etc.) and this is supported and encouraged.”* (M. J. C. Gordon 2000, p. 174)

Derived definitional principles (high-level principles programmed in ML:

- *“Melham implemented a derived type-definition principle that converts descriptions of recursive datatypes into primitive definitions and then automatically derives the natural induction and primitive recursion principles for the datatype.”* (M. J. C. Gordon 2000, p. 174)
- Melham also implemented a derived definitional principle that *“allows inductively defined relations to be specified by a transition system, and then a rule-induction tactic to be automatically generated”* (M. J. C. Gordon 2000, p. 174)
- Slind implemented a derived definitional principle for handling general recursive definitions of functions.

Lawrence C. Paulson (1986). “Natural Deduction as Higher-Order Resolution”.

Lawrence C. Paulson (1989). “The Foundation of a Generic Theorem Prover”.

Isabelle

Lawrence C. Paulson (1986). “Natural Deduction as Higher-Order Resolution”.

Lawrence C. Paulson (1989). “The Foundation of a Generic Theorem Prover”.

Isabelle is a generic interactive proof assistant for a broad class of logics.

Isabelle

Lawrence C. Paulson (1986). “Natural Deduction as Higher-Order Resolution”.

Lawrence C. Paulson (1989). “The Foundation of a Generic Theorem Prover”.

Isabelle is a generic interactive proof assistant for a broad class of logics.

The meta-logic (logic framework) of Isabelle is an intuitionistic higher-order logic.

Isabelle

Lawrence C. Paulson (1986). “Natural Deduction as Higher-Order Resolution”.

Lawrence C. Paulson (1989). “The Foundation of a Generic Theorem Prover”.

Isabelle is a generic interactive proof assistant for a broad class of logics.

The meta-logic (logic framework) of Isabelle is an intuitionistic higher-order logic.

Isabelle	{	Isabelle/CTT	(extensional constructive type theory)
		Isabelle/FOL	(first-order logic)
		Isabelle/ZF	(Zermelo-Fraenkel set theory)
		Isabelle/HOL	(higher-order logic)

Tobias Nipkow, Lawrence C. Paulson and Markus Wenzel (2002). “Isabelle/HOL. A Proof Assistant for Higher-Order Logic”.

References

-  Peter B. Andrews [1986] (2002). An Introduction to Mathematical Logic and Type Theory: To Truth Through Proof. 2nd ed. Vol. 27. Applied Logic Series. Kluwer (cit. on p. 12).
-  Alonzo Church (1940). A Formulation of the Simple Theory of Types. The Journal of Symbolic Logic 5.2, pp. 55–68. DOI: [10.2307/2266170](https://doi.org/10.2307/2266170) (cit. on pp. 13–19).
-  Willam M. Farmer (2008). The Seven Virtues of Simple Type Theory. Journal of Applied Logic 6.3, pp. 267–286. DOI: [10.1016/j.jal.2007.11.001](https://doi.org/10.1016/j.jal.2007.11.001) (cit. on pp. 13–19).
-  H. Geuvers (2009). Proof Assistants: History, Ideas and Future. Sadhana 34.1, pp. 3–25. DOI: [10.1007/s12046-009-0001-5](https://doi.org/10.1007/s12046-009-0001-5) (cit. on pp. 8, 9).
-  Michael J. Gordon, Robin Milner and Christopher P. Wadsworth (1979). Edinburgh LCF. A Mechanised Logic of Computation. Vol. 78. Lecture Notes in Computer Science. Springer. DOI: [10.1007/3-540-09724-4](https://doi.org/10.1007/3-540-09724-4) (cit. on pp. 22–26).
-  Mike Gordon (July 1985). HOL. A Machine Oriented Formulation of Higher Order Logic. Tech. rep. 68. Computer Laboratory. University of Cambridge. URL: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-68.pdf> (cit. on pp. 29–32).

References



Mike Gordon (2000). From LCF to HOL: A Short History. In: Proof, Language, and Interaction: Essays in Honour of Robin Milner. Ed. by Gordon Plotkin, Colin Stirling and Mads Tofte. Foundations of Computing Series. MIT Press. Chap. 6. DOI: [10.7551/mitpress/5641.003.0012](https://doi.org/10.7551/mitpress/5641.003.0012) (cit. on pp. 22–26, 29–37).



Robin Milner (1972). Logic for Computable Functions. Description of a Machine Implementation. Tech. rep. Stanford Artificial Intelligence Project, Memo AIM-169. STAN-CS-72-288. Stanford University (cit. on p. 21).



Tobias Nipkow, Lawrence C. Paulson and Markus Wenzel (2002). Isabelle/HOL. A Proof Assistant for Higher-Order Logic. Vol. 2283. Lecture Notes in Computer Science. Springer. DOI: [10.1007/3-540-45949-9](https://doi.org/10.1007/3-540-45949-9) (cit. on p. 42).



Lawrence C. Paulson (1989). The Foundation of a Generic Theorem Prover. Journal of Automated Reasoning 5.3, pp. 363–397. DOI: [10.1007/BF00248324](https://doi.org/10.1007/BF00248324) (cit. on pp. 38–41).



Lawrence C. Paulson [1987] (2003). Logic and Computation. Interactive Proof with Cambridge LCF. Digitally printed version. Vol. 2. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press (cit. on pp. 27, 28).

References



Dana S. Scott [1969] (1993). A Type-Theoretical Alternative to ISWIM, CUCH, OWHY. Theoretical Computer Science 121.1–2, pp. 441–440. DOI: [https://doi.org/10.1016/0304-3975\(93\)90095-B](https://doi.org/10.1016/0304-3975(93)90095-B) (cit. on p. 20).